

SOFTWARE REQUIREMENTS SPECIFICATION

CyberShield

Multi-Modal Phishing Detection Platform

SRS Report

Version 2.0

Prepared for Academic Project Documentation

CyberShield analyzes suspicious URLs, SMS/email text, and screenshots to help users identify phishing attempts before they act.

Document Control

This Software Requirements Specification (SRS) defines the functional, quality, interface, data, security, and validation requirements for CyberShield. It is intended to guide design, development, testing, review, and future maintenance.

Table 1: Revision History

Version	Date	Owner	Description
1.0	Initial project phase	CyberShield Team	URL analysis requirements established.
2.0	Current revision	CyberShield Team	Adds SMS/email NLP analysis and screenshot phishing detection requirements.

Document Status

Baseline draft for implementation and academic review. Requirements marked mandatory are necessary for the proposed release; optional enhancements are planned only after the core workflow is stable.

Approval Roles

- Project guide: validates academic scope and documentation quality.
- Development team: confirms technical feasibility and estimates.
- Test reviewer: confirms that acceptance criteria are measurable.
- Product owner: accepts the requirements baseline and priorities.

Acknowledgement

The CyberShield team gratefully acknowledges the guidance of its project mentor, faculty members, peers, and families whose feedback and encouragement support this work. The project is shaped by the practical need to make phishing warnings understandable and available before a user clicks a link, replies to a message, or shares sensitive information.

The team also recognizes the security research community, standards bodies, and open-source contributors whose work in secure systems, natural-language processing, image analysis, and threat intelligence informs the proposed design.

Abstract

CyberShield is a multi-modal phishing detection platform. Its existing URL analysis capability is extended by two complementary services: text analysis for SMS and email content, and screenshot analysis for visual phishing indicators. The platform combines cybersecurity intelligence with natural-language processing (NLP) and image processing (IP) to provide a consistent, evidence-based risk assessment.

A user can submit a URL, paste a message, or upload a screenshot. CyberShield normalizes the input, performs modality-specific analysis, generates a risk score and explanations, preserves a privacy-aware audit record, and provides safe next actions. The proposed system is web-based, role-aware, scalable, and designed to minimize the risk of overconfident or harmful automated advice.

Table of Contents

1. Introduction	1
2. Overall Description	4
3. Specific Requirements	9
4. System Models	22
5. Verification and Acceptance	26
6. Deployment and Operations	28
7. Appendices and References	30

List of Tables

Table 1 Revision History; Table 2 User Classes; Table 3 Functional Requirements; Table 4 Non-functional Requirements; Table 5 Data Entities; Table 6 Traceability Matrix.

List of Figures

Figure 1 Platform Context; Figure 2 Processing Pipeline; Figure 3 Data Flow; Figure 4 Entity Relationship View; Figure 5 Sequence View.

1. Introduction

CyberShield addresses the growing variety of phishing channels. A link-only tool is useful but incomplete: attackers also rely on persuasive SMS messages, email bodies, and images that imitate legitimate brands. CyberShield unifies these signals into a single user-facing risk assessment while preserving the evidence needed for later review.

1.1 Purpose

The purpose of this SRS is to define what CyberShield must do, its boundaries, interfaces, data behavior, and quality attributes. It is written for developers, testers, project reviewers, and future maintainers.

1.2 Scope

The system shall allow a registered or guest user to analyze a URL, text message, email content, or screenshot. It shall identify suspicious indicators, assign a transparent risk level, give safety guidance, retain records according to policy, and support administrative oversight. It does not guarantee that every threat is detected or replace organizational incident response.

1.3 Goals

- Detect common phishing indicators across web, text, and image inputs.
- Explain findings in plain language without encouraging unsafe actions.
- Protect submitted content, especially personally identifiable information.
- Support continuous model and rule improvement using approved feedback.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

1.4 Definitions, Acronyms, and Abbreviations

Phishing: a deceptive attempt to obtain credentials, money, or sensitive information. URL: a web address. NLP: computational processing of natural-language text. IP: image processing. OCR: optical character recognition. IOC: indicator of compromise. Risk score: a normalized estimate used to classify an input as low, medium, high, or critical risk.

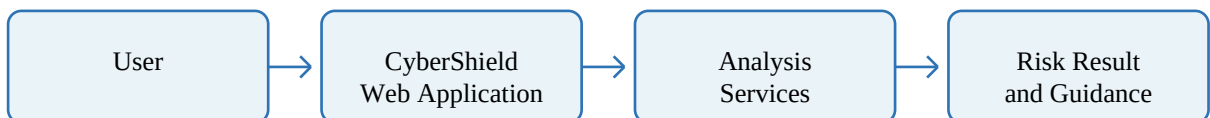
1.5 References

- IEEE 830, recommended practice for software requirements specifications.
- OWASP guidance on phishing, secure application design, and logging.
- NIST SP 800-53 security and privacy controls; NIST guidance on incident response.
- Project references: EXPIROCHAIN report structure and supplied SRS Formatting Guide.

1.6 Document Overview

Section 2 describes the product context and users. Section 3 contains requirements for each analysis mode, interfaces, quality attributes, and data. Section 4 provides system models. The final sections define verification, operations, appendices, and references.

Figure 1: CyberShield Platform Context



Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

2. Overall Description

2.1 Product Perspective

CyberShield is a web platform that coordinates a client interface, an application API, a URL intelligence service, an NLP service, an image/OCR service, a risk-scoring engine, and a protected data store. It can consume configurable reputation feeds and may later integrate an organization's security operations workflow.

The product is a new multi-modal layer rather than a browser replacement, mail gateway, or antivirus tool. It performs pre-action triage: it helps a user decide whether to avoid, verify, report, or safely continue.

2.2 Product Functions

- Submit and normalize URLs for lexical, DNS/reputation, redirect, and certificate-related checks.
- Extract semantic and structural phishing cues from pasted SMS and email content.
- Extract visible text and visual cues from screenshots using OCR and image analysis.
- Fuse modality signals into an explainable classification and record an audit trail.
- Present recommendations, feedback controls, administrative configuration, and reports.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

2.3 User Classes

Table 2: User Classes and Access

Class	Description	Primary permissions
Guest user	Unregistered visitor	Submit limited analyses; view current result only.
Registered user	Individual account holder	Analyze, view history, submit feedback, manage privacy settings.
Analyst	Authorized security reviewer	Review flagged cases and feedback; annotate disposition.
Administrator	Platform operator	Manage users, policies, integrations, and model/rule releases.
Auditor	Read-only oversight role	View security-relevant logs and compliance reports.

2.4 Operating Environment

The client shall support current desktop and mobile versions of major browsers. The server shall run in a hardened cloud or institutional environment using HTTPS. Processing services may use containerized Python/ML workloads and a relational database; object storage may retain approved screenshot artifacts.

2.5 Design and Implementation Constraints

- All data in transit shall use TLS; insecure HTTP callbacks are prohibited.
- The UI shall never render a submitted URL as an automatically clickable link before warning the user.
- Analysis models must expose a version identifier with each result.
- Image size, file type, retention, and throughput shall be constrained to reduce abuse.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

2.6 Assumptions and Dependencies

CyberShield assumes that users submit content they are authorized to inspect and that external reputation sources can be unavailable, stale, or incomplete. A network connection is required for intelligence lookups. OCR accuracy depends on screenshot resolution, language, compression, and visual complexity.

Dependencies include an authentication provider, URL/domain reputation feeds, an OCR engine, an NLP classifier or ruleset, secure object storage, database backups, and operational monitoring. A failed dependency shall degrade a result with a visible limitation instead of silently producing certainty.

2.7 Out of Scope

- Automatic clicking, visiting, or credential submission to suspicious websites.
- Forensic compromise confirmation on the user device.
- Blocking messages at a telecom carrier or enterprise email gateway.
- Legal adjudication or irreversible account actions based solely on a model score.

2.8 Business Rules

The system shall classify results as Low, Medium, High, or Critical. High and Critical results must display a prominent safety warning. A result can be “Inconclusive” when evidence is insufficient or a dependency is unavailable. Users may delete their history subject to legal and institutional retention requirements.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

3. Specific Requirements

3.1 Requirement Conventions

Each mandatory requirement uses the form FR (functional requirement) or NFR (non-functional requirement). “Shall” indicates a required behavior. Priority labels are Must, Should, or Could. Acceptance statements are intentionally testable.

3.2 URL Analysis Requirements

FR-URL-01 (Must): The system shall accept a single URL as text and reject malformed, empty, or oversize input with a clear message.

FR-URL-02 (Must): The system shall normalize the URL, parse hostname and path components, and display the normalized value used for analysis.

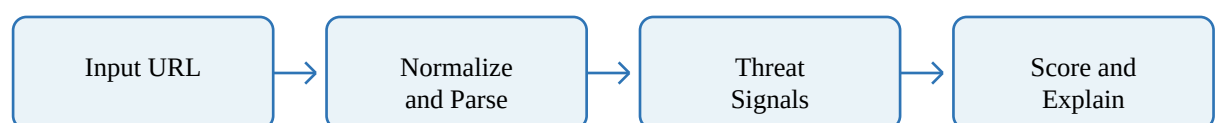
FR-URL-03 (Must): The system shall evaluate lexical indicators including misleading subdomains, IP-address hosts, excessive length, encoded characters, suspicious keywords, and homoglyph risk.

FR-URL-04 (Must): The system shall query configured reputation and threat-intelligence sources without opening the target in the user’s browser.

FR-URL-05 (Should): The system shall assess redirect-chain evidence through a sandboxed service where approved by policy.

FR-URL-06 (Must): The result shall show risk level, confidence band, indicators, limitations, and safe next actions.

Figure 2: URL Processing Pipeline



3.3 SMS and Email Text Analysis Requirements

FR-TXT-01 (Must): The system shall accept pasted SMS text, email subject/body text, or structured email fields when available.

FR-TXT-02 (Must): The system shall remove or mask configured personally sensitive fields before persistent storage while preserving sufficient evidence for scoring.

FR-TXT-03 (Must): The NLP pipeline shall identify urgency, threats, reward promises, impersonation cues, credential/payment requests, suspicious links, and grammar or formatting anomalies.

FR-TXT-04 (Must): The system shall extract embedded URLs, phone numbers, email addresses, currency/payment references, and named brands when present.

FR-TXT-05 (Must): The system shall compare claimed sender identity with content claims and flag apparent brand impersonation when confidence exceeds the configured threshold.

FR-TXT-06 (Should): The system shall support English initially and return a limitation notice for unsupported or mixed-language input.

FR-TXT-07 (Must): The result shall quote only minimal, safely masked evidence snippets and shall not expose secrets detected in the submission.

Implementation Notes

Implementation note: Text analysis must preserve the difference between a suspicious expression and a confirmed malicious event. The NLP service identifies cues such as urgency, impersonation, credential requests, payment pressure, and unusual sender claims, then links those cues to the exact masked evidence used in the explanation. It also records language and OCR confidence where applicable, allowing future evaluation to identify cases in which wording, dialect, or short message format affects reliability.

3.4 Screenshot Phishing Detection Requirements

FR-IMG-01 (Must): The system shall accept PNG, JPEG, and WEBP screenshots within configured size and resolution limits.

FR-IMG-02 (Must): The system shall validate file signature and decode image pixels before analysis; files that fail validation shall be rejected.

FR-IMG-03 (Must): The image pipeline shall run OCR to obtain visible text and route that text through the same text-analysis service where applicable.

FR-IMG-04 (Must): The pipeline shall detect visual phishing signals including login forms, password/OTP prompts, payment prompts, brand-like logos, shortened/obscured links, QR codes, and urgency banners.

FR-IMG-05 (Should): The system shall compare extracted domain/brand clues with known legitimate domain patterns and identify likely lookalikes.

FR-IMG-06 (Must): The result shall identify which evidence originated from OCR, visual analysis, or linked text analysis, and shall state OCR confidence.

FR-IMG-07 (Must): The system shall strip executable metadata and never execute embedded content.

Implementation Notes

Implementation note: Image analysis is designed as a layered process. File validation occurs before OCR or visual classification; decoded pixels, not browser-rendered content, are used for analysis. OCR text, form-like layout, QR-code presence, and logo-like visual patterns are separate evidence sources. The final explanation names their origin and confidence so a user understands when a conclusion relies on clearly readable text versus a weaker visual resemblance.

3.5 Cross-Modal Scoring and Results

FR-SCR-01 (Must): The scoring engine shall combine applicable indicators using configurable, versioned weights or an approved model.

FR-SCR-02 (Must): The engine shall avoid double-counting the same extracted URL or claim across modalities.

FR-SCR-03 (Must): Each result shall include a score from 0 to 100, risk label, confidence band, reasons, model/rule version, and analysis timestamp.

FR-SCR-04 (Must): A Critical result shall recommend that the user avoid interacting with the content, verify through an independently known channel, and report according to local policy.

FR-SCR-05 (Must): An Inconclusive result shall identify unavailable sources or low-quality evidence and recommend a safer verification path.

FR-SCR-06 (Should): An authorized analyst may override a classification with an explanation; overrides shall be immutable audit events.

Risk Interpretation

0-24 Low: no strong known indicators. 25-49 Medium: caution warranted. 50-74 High: likely suspicious. 75-100 Critical: strong indicators or known malicious evidence. Scores are decision support, not proof of fraud.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

3.6 Account, History, and Feedback Requirements

FR-ACC-01 (Must): Registered users shall be able to create an account, sign in, sign out, reset credentials through the approved identity mechanism, and manage consent preferences.

FR-ACC-02 (Must): The system shall display analysis history only to its owner or an authorized role.

FR-ACC-03 (Must): A user shall be able to delete an individual history item or request deletion of eligible history records.

FR-FDB-01 (Must): Users shall be able to mark a result as helpful, unhelpful, suspected false positive, or suspected false negative.

FR-FDB-02 (Should): Feedback intended for learning shall be reviewed or de-identified before it is used to tune a model or rule.

FR-RPT-01 (Must): The system shall generate a shareable incident summary containing safe, masked indicators and no raw credentials.

FR-RPT-02 (Should): The system shall export authorized summaries in PDF or CSV format with an export audit event.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

3.7 Administration and Governance Requirements

FR-ADM-01 (Must): Administrators shall manage user roles, retention periods, permitted file types, maximum file sizes, and enabled intelligence sources.

FR-ADM-02 (Must): Administrators shall manage scoring thresholds and model/rule releases through a change-controlled configuration process.

FR-ADM-03 (Must): The system shall preserve a versioned audit event for administrative changes, analyst overrides, exports, deletion actions, and sign-in failures.

FR-ADM-04 (Should): Administrators shall view aggregated metrics for volume, risk distribution, processing failures, feedback trend, and service latency.

FR-ADM-05 (Must): Only authorized administrators shall access raw retained screenshots, and all such access shall be logged.

FR-ADM-06 (Must): The administrative interface shall provide a health view that distinguishes service failure, rate limiting, model unavailability, and intelligence-feed degradation.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

3.8 External Interface Requirements

User Interface

The interface shall offer clearly separated Analyze URL, Analyze Text, and Analyze Screenshot actions. Upload and submit buttons shall state what data will be processed. The result shall show a risk badge, an evidence list, limitations, and safe actions. Color shall not be the only way a risk level is conveyed.

Software Interfaces

The API shall expose authenticated endpoints for submission, result retrieval, history, feedback, and administration. It shall use JSON over HTTPS. External feeds shall be accessed through server-side adapters so credentials and provider-specific behavior are not exposed to clients.

Hardware Interfaces

No special client hardware is required. Camera capture is outside the baseline; users upload existing images. Servers may use CPU or approved GPU workers for OCR and image models.

Communications Interfaces

The service shall use HTTPS with modern TLS. SMTP is not required in the baseline. Notifications, if added, shall use a vetted provider and never include raw suspicious content in a notification preview.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

3.9 Performance Requirements

NFR-PERF-01 (Must): For a typical URL or text submission, the system shall return an initial result within 5 seconds at the 95th percentile under the defined baseline load.

NFR-PERF-02 (Must): For a screenshot up to the configured maximum, the system shall return a result within 15 seconds at the 95th percentile, excluding a displayed external-feed timeout.

NFR-PERF-03 (Must): The system shall support at least 50 concurrent interactive analyses in the academic deployment baseline without data cross-contamination.

NFR-PERF-04 (Should): Long-running tasks shall be queued and expose a request status rather than holding a browser connection indefinitely.

NFR-PERF-05 (Must): A result page shall render with accessible core content before optional charts or enrichment complete.

Capacity and Limits

Initial limits are: URL 4,096 characters; text 25,000 characters; screenshot 10 MB; history retention configurable from 0 to 180 days. Limits are configurable administrative controls and must be visible in the UI.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

3.10 Security and Privacy Requirements

NFR-SEC-01 (Must): The system shall enforce role-based access control using least privilege.

NFR-SEC-02 (Must): The system shall encrypt data in transit and encrypt stored sensitive data using approved platform controls.

NFR-SEC-03 (Must): The system shall protect against common web threats including injection, cross-site scripting, request forgery, insecure file upload, and broken authorization.

NFR-SEC-04 (Must): Credentials, API keys, session tokens, and detected secrets shall not appear in application logs or result pages.

NFR-SEC-05 (Must): Screenshot uploads shall be malware-scanned where infrastructure permits, stored outside the web root, and accessed through time-limited authorization.

NFR-PRV-01 (Must): The system shall minimize collection, explain retention at submission, and honor approved deletion requests.

NFR-PRV-02 (Must): Analytics shall use aggregated or de-identified data where practical.

NFR-SEC-06 (Should): High-risk administrator actions shall require re-authentication or multi-factor authentication.

Implementation Notes

Implementation note: Security controls are evaluated as part of normal product behavior rather than treated as a final deployment task. Each access decision, sensitive export, analyst override, and configuration change must be attributable to an actor and a correlation identifier. Retention settings apply to raw input, derived indicators, assets, and backups separately. This design supports review without retaining more personal content than is necessary for the platform purpose.

3.11 Reliability, Usability, and Accessibility

NFR-REL-01 (Must): The service shall fail safely. When a dependency fails, it shall identify the limitation and not label content safe solely because a lookup failed.

NFR-REL-02 (Must): Daily backups of required structured data shall be verifiable, and restoration shall be tested at least once per release cycle.

NFR-USE-01 (Must): The result explanation shall use concise, non-technical wording with expandable technical detail.

NFR-USE-02 (Must): The system shall not use fear-inducing or absolute language such as “guaranteed safe.”

NFR-ACC-01 (Must): Keyboard navigation, visible focus, text alternatives, sufficient contrast, and readable risk labels shall be supported.

NFR-MNT-01 (Must): Rules, model artifacts, and integrations shall be versioned and traceable to a release record.

NFR-LOC-01 (Could): Future releases may support additional languages, locale-aware phishing examples, and right-to-left layouts.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

3.12 Data Requirements

Table 5: Core Data Entities

Entity	Key fields	Purpose
User	user_id, role, consent, created_at	Identity and authorization.
AnalysisRequest	request_id, modality, submitted_at, status	Tracks an analysis job.
AnalysisResult	result_id, score, label, version, created_at	Stores outcome and evidence summary.
Indicator	indicator_id, type, value_hash, severity	Stores a normalized signal safely.
ScreenshotAsset	asset_id, object_ref, hash, OCR_confidence	Controlled image metadata.
Feedback	feedback_id, result_id, category, note	Captures user feedback.
AuditEvent	event_id, actor, action, timestamp	Immutable governance evidence.

Data Lifecycle

Raw input is accepted, validated, analyzed, and either discarded or retained according to consent and policy. Derived indicators are preferred over raw content for analytical history. Screenshot storage uses a separate protected object reference; the database stores only the minimum link and integrity hash.

Implementation Notes

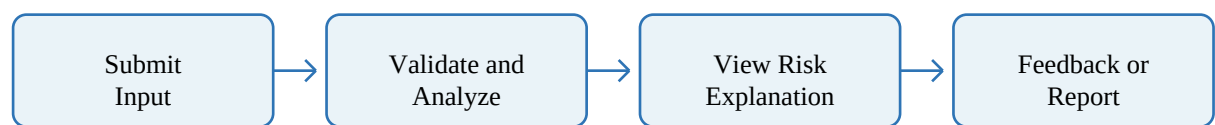
Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

4. System Models

4.1 Use Case Model

Primary actor: User. Supporting actors: Analyst, Administrator, External Intelligence Provider, OCR Service. The user submits one of the three supported modalities and receives a result; the analyst reviews flagged cases; the administrator configures policy and inspects operational health.

Figure 3: High-Level Use Case Flow



4.2 Model Assumptions

Models identify information flow and responsibility; they do not prescribe a specific framework or database vendor. All inter-service calls are authenticated and observable.

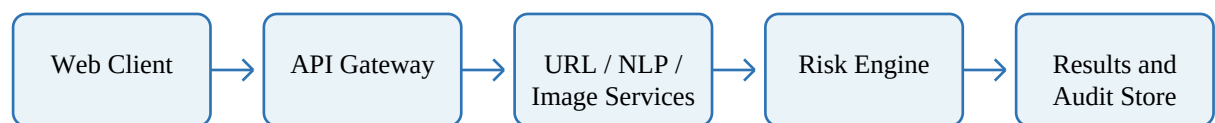
Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

4.3 Data Flow Model

The client sends input to the API gateway. The gateway validates identity, rate limits, and input type. It routes URL, text, and image requests to the relevant processor. The score engine aggregates normalized indicators; the result service stores an approved record and returns an explanation. External feeds provide enrichments but never receive unnecessary user identity data.

Figure 4: Data Flow



4.4 Trust Boundaries

The browser, application tier, worker tier, external feeds, and storage tier are distinct trust zones. Input validation occurs at the boundary. Secrets remain server-side. Raw assets never cross into public static hosting.

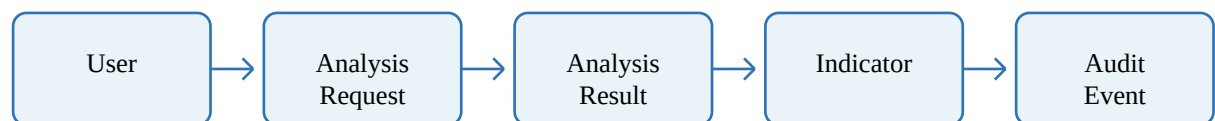
Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

4.5 Entity Relationship View

A User owns zero or more AnalysisRequests. Each request produces zero or one AnalysisResult during processing; a result contains one or more Indicators and can receive zero or more Feedback records. ScreenshotAsset is optional and linked only for image modality. AuditEvent references an actor and a protected action target.

Figure 5: Entity Relationship View



4.6 Sequence View

The submit sequence is: user submits input; client receives a request identifier; gateway validates and dispatches; processors produce evidence; scoring creates the decision; the result is persisted; the client displays the explanation. If a source times out, the processor returns an explicit unavailable signal and scoring applies the uncertainty policy.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

4.7 State Model

AnalysisRequest states are Created, Validating, Queued, Processing, Completed, Inconclusive, Failed, and Deleted. A request may transition to Inconclusive after permitted enrichment timeouts; it may transition to Failed only for technical or invalid-input reasons. Completed results remain immutable except for an authorized analyst disposition or user feedback, each recorded separately.

4.8 Error Handling Model

Invalid input yields a client-safe validation error. A temporary service failure yields a retryable response with a correlation identifier. A policy violation yields a clear explanation without disclosing security-control internals. Security events are logged in a protected audit channel.

4.9 Risk Decision Model

The decision model uses evidence quality, severity, corroboration, and source reliability. It must distinguish absent evidence from unavailable evidence. A high score from one reliable malicious-domain source may be sufficient, whereas weak linguistic signals alone should yield cautious wording and appropriate confidence.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

5. Verification and Acceptance

5.1 Requirements Validation

Validation shall combine review, automated tests, integration tests, security tests, usability checks, and controlled test datasets. Test data shall not include live malicious payloads, active credentials, or unauthorized personal information.

Table 6: Sample Traceability Matrix

Requirement	Verification method	Acceptance evidence
FR-URL-03	Unit and integration tests	Indicators returned for curated URLs.
FR-TXT-03	NLP test corpus and review	Urgency and credential-request cues identified.
FR-IMG-03	OCR fixture tests	Expected text and confidence recorded.
FR-SCR-03	API contract test	Score, label, reasons, version returned.
NFR-SEC-05	Security review and test	Rejected unsafe upload; protected asset path.
NFR-PERF-02	Load test	95th percentile screenshot latency report.

5.2 Acceptance Criteria

- All Must requirements have passing verification evidence or an approved, documented exception.
- No Critical or High result displays a clickable suspicious link by default.
- No test reveals raw credentials or secrets in UI, logs, or export.
- The final demo can analyze representative URL, text, and screenshot cases end to end.

Implementation Notes

Implementation note: Verification evidence should demonstrate both successful and degraded behavior. Test cases therefore include valid samples, malformed inputs, unavailable feeds, low-quality screenshots, denied authorization requests, and scoring-policy changes. The team records expected result labels as well as expected limitations, because a system that explains an unavailable dependency correctly is safer than one that silently produces an apparently certain assessment.

5.3 Test Strategy

Unit tests cover parsers, normalizers, score calculations, masking, authorization rules, and storage adapters. Integration tests cover API-to-worker flow, feed timeouts, OCR handoff, and audit events. End-to-end tests use approved benign and synthetic phishing examples. Performance tests measure concurrent submissions and queue behavior.

5.4 Security Testing

Security testing includes dependency review, authentication and authorization tests, upload validation, API schema validation, rate-limit tests, logging inspection, and a review against relevant OWASP risks. Testing must be authorized and confined to owned systems.

5.5 Model Evaluation

The project shall record precision, recall, false-positive rate, false-negative rate, and latency for representative labeled datasets. Scores must be interpreted with dataset limitations. Model updates require regression testing against a held-out approved set and review of changed error patterns.

5.6 Usability Evaluation

At least a small representative user review shall verify that users can submit each modality, understand the risk label, find the evidence, and identify a safe next step. Feedback must not be used to claim statistical validity unless the study design supports it.

Implementation Notes

Implementation note: Verification evidence should demonstrate both successful and degraded behavior. Test cases therefore include valid samples, malformed inputs, unavailable feeds, low-quality screenshots, denied authorization requests, and scoring-policy changes. The team records expected result labels as well as expected limitations, because a system that explains an unavailable dependency correctly is safer than one that silently produces an apparently certain assessment.

6. Deployment and Operations

6.1 Deployment Architecture

A recommended deployment separates the browser client, application API, asynchronous worker queue, URL/NLP/image processors, database, protected object storage, and monitoring stack. Environment-specific configuration and secrets are injected securely; they are never committed to source control.

6.2 Monitoring and Logging

Operational metrics include request count by modality, success/error rate, latency percentiles, queue depth, dependency status, model version distribution, and storage growth. Security logs include authentication failures, authorization denials, administrative changes, sensitive exports, and protected asset access. Logs must be integrity protected and retention limited.

6.3 Incident Handling

When CyberShield itself is suspected of compromise, administrators shall contain access, preserve authorized evidence, rotate relevant credentials, assess affected data, restore service safely, and document the event. Users should be informed through approved channels when disclosure is required.

6.4 Maintenance

Routine maintenance covers dependency patches, threat-feed health, model drift checks, backup verification, capacity review, and periodic access review. A rollback path is required for scoring rules and model releases.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

7. Appendices

Appendix A: Sample Result Structure

A result contains: request identifier; modality; normalized input summary; risk score and label; confidence; detected indicators with severity and evidence source; safe recommendations; limitations; created time; rule/model version; and retention/deletion controls.

Appendix B: Safe User Guidance

- Do not click or reply when a message demands urgent credential, OTP, or payment action.
- Verify using a trusted official app, bookmarked site, or independently known phone number.
- Do not share passwords, one-time codes, recovery codes, or card details.
- Report suspicious content through the appropriate organization or service channel.

Appendix C: Future Enhancements

Potential future work includes multilingual NLP, QR-code destination analysis, email-header parsing, explainability improvements, organization-specific brand profiles, browser extension integration, and SOC/ticketing integration. Each enhancement requires a separate privacy and security assessment.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

Appendix D: Functional Requirement Summary

Table 7: Requirement Groups

Group	Purpose	Priority
URL analysis	Detect technical and reputation-based URL risk indicators.	Must
Text analysis	Identify phishing intent and entities in SMS/email text.	Must
Screenshot analysis	Combine OCR and visual cues to assess phishing screenshots.	Must
Scoring	Produce an explainable, calibrated risk decision.	Must
Governance	Control roles, configuration, audit, and retention.	Must
Reporting	Provide safe summaries, exports, and user feedback.	Should

Interpretation Note

The requirements are deliberately layered: validation and safe storage are prerequisites for analysis; modality-specific evidence is normalized before it reaches scoring; the result experience then explains uncertainty and recommends a safe action. This approach makes each component testable without treating any one signal as definitive.

Configuration Baseline

Thresholds, enabled feeds, retention duration, supported languages, maximum inputs, and model versions are configuration values. They must be change controlled and recorded with each analysis result so the behavior can be reproduced during review.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

Appendix E: Example Indicators

URL Indicators

- Domain contains a suspicious lookalike of a known brand.
- Hostname is a raw IP address or uses misleading nested subdomains.
- URL includes excessive encoding, unusual path structure, or credential/payment terms.
- Configured intelligence source reports a malicious or suspicious reputation.

Text Indicators

- Artificial urgency, account threat, prize, refund, or unexpected payment language.
- Request for password, OTP, recovery code, card detail, or remote-access action.
- Impersonation claim that conflicts with sender clues or embedded domain.
- Obscured link, unusual contact channel, or abrupt instruction to move conversation.

Image Indicators

- OCR extracts a login form, password/OTP request, or payment prompt.
- Visual structure imitates a brand while domains or text show mismatches.
- QR code or shortened link attempts to conceal destination information.
- Low image quality reduces OCR confidence and must reduce certainty.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

Appendix F: Risk Communication Examples

Low Risk

“No strong phishing indicators were found in this analysis. This does not confirm that the content is safe. Continue only if you expected it and can verify the sender or website independently.”

Medium Risk

“This content includes signals often used in scams, such as urgency or an unfamiliar link. Do not provide credentials or payment information. Verify through an official channel before taking action.”

High / Critical Risk

“Multiple strong phishing indicators were detected. Avoid clicking, replying, scanning a QR code, or sharing information. Use a known official website or contact method to verify the request and report it if appropriate.”

Inconclusive

“The analysis could not use one or more evidence sources, or the image/text quality was limited. Treat the content cautiously and verify through an independently trusted channel.”

These examples are guidance templates, not legal or incident-response instructions. The final user interface shall show the specific reasons behind each classification.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

8. Detailed Architecture Requirements

8.1 Client and API Layer

The browser client is responsible for collecting input, presenting consent language, showing progress, and rendering the completed assessment. It must not execute URLs, decode unknown files through browser plug-ins, or embed untrusted submitted content as active HTML. The API layer is responsible for authentication, validation, rate limiting, request identifiers, routing, response shaping, and audit correlation. It is deliberately kept separate from resource-intensive analysis workers so a high-cost screenshot cannot prevent normal account or result operations.

8.2 Asynchronous Worker Layer

Analysis workers consume validated jobs from a queue. A job contains a request identifier, an input reference, a modality, a privacy policy identifier, and a trace context; it does not require the worker to know the user password or browser session. Workers return normalized indicators, evidence confidence, processing status, and an analyzer version. The score engine receives these normalized outputs and applies a consistent decision policy. This separation permits independent scaling of URL, NLP, and image workloads.

8.3 Storage Layer

Structured records are kept in the database with foreign-key and authorization controls. Large image assets are stored in protected object storage under an opaque object reference. Access to an object must be mediated by an authorization check and a short-lived retrieval mechanism. A content hash supports integrity checks and duplicate detection without retaining unnecessary copies. Backups must be encrypted, tested, and governed by the same retention schedule as the primary store.

Implementation Notes

Implementation note: This requirement group is implemented through explicit interfaces, versioned configuration, and measurable acceptance criteria. The design keeps input validation, evidence collection, risk scoring, explanation, and audit recording distinct so that a change in one component can be reviewed and tested without unintentionally changing every result. Where uncertainty exists, the platform states the limitation and provides a safe verification action instead of presenting a binary assurance.

8.4 URL Analysis Design Detail

URL analysis begins with deterministic parsing. The system records scheme, hostname, effective top-level domain, path, query, port, and selected normalized forms. It identifies transformations that conceal intent, such as percent-encoding, repeated delimiters, embedded user-info fields, punycode, or a long path crafted to hide the real host. The parser shall not treat a visual resemblance as an established identity; it reports it as a lookalike indicator with an explanation.

The reputation adapter queries approved sources by domain, URL, hash, or IP according to the source contract. Timeouts and provider errors are represented as source-unavailable indicators. A provider response has a source name, fetch time, policy-approved confidence, and response classification. Cached reputation data is marked with age so a user and analyst can understand whether evidence is fresh. The platform must never transmit a user identifier, personal note, or full unrelated message body to a URL intelligence provider.

When sandboxed redirect evaluation is enabled, the worker uses isolated networking and strict time, size, and hop limits. The returned output contains only the final normalized domains, redirect count, and selected safety indicators. It must not download unknown executable content, interact with forms, or attempt credential submission. These controls make URL enrichment an observation process rather than an active browsing action.

8.5 NLP Analysis Design Detail

The text service first separates message parts into subject, sender label, body, quoted reply, and attachment description when those fields are present. It normalizes whitespace, common obfuscation, Unicode variants, and repeated punctuation while retaining an original protected copy only when policy permits. Tokenization and sentence segmentation provide input to rules and classifier features. The service detects requests for secrets, money, account verification, delivery confirmation, remote access, or rapid action; it also detects coercive language such as threats of account closure.

Entity extraction identifies URLs, email addresses, telephone numbers, payment handles, organization names, dates, and amounts. A claimed organization is compared with text surrounding the request and any embedded domains. The comparison produces an explanation such as “message claims to be from a bank but directs you to an unrelated domain,” rather than an unsupported accusation. Where confidence is low, the result says that the wording is unusual or cannot be verified.

NLP output is not a substitute for human language understanding in every context. Legitimate operational messages may use urgent wording, and phishing messages may be grammatically polished. Therefore, the engine combines linguistic evidence with link, sender, or image evidence, applies conservative labels to isolated weak cues, and captures evaluation metrics by language and content type.

8.6 Image and OCR Analysis Design Detail

Screenshot analysis starts with file validation and pixel decoding. The service checks the declared type against the file signature, measures dimensions, and rejects image bombs or unsupported color formats. It removes nonessential metadata from the working derivative. The protected original is retained only if consent and retention policy allow it. A thumbnail used by analysts must be generated from the safe decoded pixels rather than served directly from the original upload.

OCR produces text blocks, confidence values, and approximate coordinates. The text blocks feed the NLP service, while layout logic uses coordinates to recognize forms, input labels, submit buttons, QR codes, browser-like address bars, and warning banners. Visual detection may also estimate whether a logo or color pattern appears brand-like; it cannot establish ownership or authenticity by itself. Such cues are scored as corroborative observations.

The user result shall make the provenance of each signal clear. For example, “OCR detected the phrase ‘enter your one-time code’ with high text confidence” is distinct from “the image contains a logo-like graphic.” If OCR confidence is poor, the platform lowers the certainty of text-derived conclusions and prompts the user to verify manually rather than creating a falsely precise score.

8.7 Risk-Scoring Methodology

The scoring service is an explicit policy component. It receives normalized indicators with source reliability, severity, freshness, modality, and confidence. Before calculation it merges duplicate evidence: a URL extracted from a screenshot and the same URL pasted in the text is one indicator with multi-modal corroboration, not two unrelated threats. Severity weights represent potential harm and evidence reliability; confidence reflects how well a signal was observed.

A rules-first baseline is suitable for the academic release because it can be reviewed and tested deterministically. A future trained model may provide a bounded score contribution when it has documented training data, evaluation results, drift monitoring, and a rollback path. In either approach, thresholds are versioned configuration. A decision record must preserve the score inputs, the applied policy version, and the final label so retrospective review is possible.

Calibration checks are required. A score of 70 must have a stable meaning across modalities and must not result merely from a long message containing many weak features. The team shall review false positives and false negatives by modality, adjust thresholds under change control, and test representative clean, ambiguous, and malicious examples before release.

8.8 User Experience and Safety Content

The first screen should explain that CyberShield analyzes submitted material for signs of phishing and that no automated tool can prove complete safety. The three analysis methods are presented as equal choices so users are not led to assume phishing exists only in URLs. Submission controls include clear examples, input limits, privacy information, and a visible option to remove an uploaded asset after the current analysis when policy allows.

A result page has four reading levels: the risk label and immediate safe action; a short explanation; a detailed list of indicators; and technical metadata for authorized users. High-risk pages keep the safe action near the top: avoid clicking, do not enter credentials or codes, verify using an independently known channel, and report through a suitable process. The visual design uses text labels and icons as well as color. It does not use panic language, flashing warnings, or a false “safe” badge.

When a user supplies feedback, the form distinguishes “the explanation was unclear” from “the classification seems wrong.” It warns users not to add passwords, OTPs, banking information, or personal identity documents in the feedback note. This avoids turning an improvement pathway into a secondary data-collection risk.

8.9 Authorization and Audit Detail

Authorization is evaluated on every server-side action. A registered user can see only their own current and historical analyses. An analyst can access assigned or policy-authorized flagged cases but does not automatically receive administration rights. Administrators configure the system but are still subject to audit when viewing sensitive artifacts. Auditors receive a read-only view of permitted logs and reports. The role model must deny access by default and use server-side checks, not only hidden user-interface controls.

Audit events record the actor identifier or service identity, event type, target type, target identifier, time, correlation identifier, result, and limited contextual metadata. They do not record passwords, raw session tokens, or unrestricted original message content. Events are append-only from the application point of view; changes to audit storage are restricted and monitored. The audit trail supports investigation of configuration changes, access to raw screenshots, exports, deletions, analyst overrides, and repeated sign-in failures.

The service accounts used by OCR, reputation, and storage adapters have minimum permissions. Secrets are stored in an approved secret manager or protected deployment configuration and are rotated on a planned schedule or immediately after suspected exposure.

8.10 Privacy, Retention, and Ethical Use

CyberShield processes content that can contain private conversations, contact details, financial references, and screenshots of accounts. The product therefore uses data minimization: only input required for the selected analysis is collected, raw data is retained only when permitted, and derived evidence is preferred for history and metrics. A clear notice explains that submitted content may be inspected by authorized personnel if the user consents or policy requires it.

Retention is a configurable policy rather than a hidden implementation detail. The baseline distinguishes transient processing artifacts, user-visible history, aggregated metrics, and protected security audit records. Each class has a purpose, owner, retention duration, deletion method, and approved exceptions. On expiration, stored raw assets and content are deleted or irreversibly de-identified. Backup expiration is documented so deletion requests are handled consistently.

The project must avoid using private user submissions to train models by default. If a future learning program is approved, it should use de-identified, consented, reviewed data with access restrictions. Human reviewers should see the least information required to evaluate a case and should receive guidance on handling sensitive or distressing content.

8.11 Detailed Test Cases

Table 8: Representative Acceptance Cases

Case	Input and action	Expected outcome
TC-01	Paste a malformed URL.	Validation message; no analysis job is created.
TC-02	Submit a known-safe synthetic domain.	Low or inconclusive result with stated limitations; no automatic navigation.
TC-03	Submit synthetic SMS requesting an OTP via a suspicious link.	Text cues, extracted link, High/Critical guidance, masked evidence.
TC-04	Upload a synthetic login screenshot with a QR code.	File validates; OCR and visual indicators reported with confidence.
TC-05	Force a reputation adapter timeout.	Result is inconclusive or reduced-confidence; timeout is explained.
TC-06	Attempt cross-user history access.	Server returns authorization denial and audit event.

Test data will use fictional domains, synthetic messages, and authorized image fixtures. No test shall distribute active malicious links, credential harvesters, or private user data. Regression suites should run for every scoring-policy, model, parser, or dependency update.

8.12 Deployment, Release, and Rollback

The release process separates development, test, staging, and production-like environments. Configuration values and external credentials differ by environment and must never be copied into source code. Before release, the team reviews unit and integration results, dependency vulnerabilities, schema migrations, model/rule version, performance evidence, backup status, and operational runbook updates.

A release is reversible. Rule and model updates are labeled with a unique version, tested against an approved regression set, and deployed in a way that allows the prior version to be restored. Database migrations include a backup and rollback decision. Monitoring thresholds are checked immediately after deployment for request errors, latency, queue depth, unexpected score distribution, and external-feed failures.

Changes that affect personal-data retention, analyst access, or third-party sharing require privacy and security review before deployment. Emergency changes follow a documented exception path, including a post-change review and complete audit record.

8.13 Operational Runbook Summary

Normal operations begin with health checks for the API, queue, processors, database, object storage, and intelligence adapters. A degraded external feed is not automatically a platform outage; the system continues with a visible limitation if the remaining evidence is sufficient. A database, authentication, or storage authorization failure is handled as a service-impacting event because it may affect safety or privacy.

When a worker backlog grows, operators check input volume, processing latency, error rate, worker availability, and unusually large image submissions. Capacity can be increased by adding isolated worker instances subject to rate limits and cost controls. Operators do not inspect raw user inputs unless that access is necessary, authorized, and logged.

The runbook includes contacts, escalation conditions, backup restoration steps, model/rule rollback steps, security incident handling, and a user-facing status communication template. Periodic drills validate that the team can restore a backup, disable a faulty intelligence adapter, and recover from a failed model release without exposing data or returning misleading “safe” decisions.

8.14 Expanded Glossary

Brand impersonation: use of a name, image, or wording intended to appear associated with an organization. Confidence: an estimate of evidence quality, not the probability that a user will be harmed. Domain: the registered internet name used to identify a host. Homoglyph: a character that visually resembles another character. Lookalike domain: a domain crafted to resemble a legitimate one. OCR confidence: the engine's estimate that extracted image text is correct.

Protected asset: a stored uploaded image available only through authorized retrieval. Reputation feed: an external source that provides reported risk information about an indicator. Risk label: the user-facing category derived from score and policy. Sandboxed evaluation: constrained inspection performed in an isolated environment. Signal provenance: the source and method through which evidence was obtained. Threat intelligence: curated data about malicious infrastructure or behavior.

User-visible explanation: short wording that states why the result was produced and what to do next. Versioned policy: a named, immutable release of thresholds, weights, and decision rules. This glossary is normative where a term is used in a requirement; implementation documentation may add vendor-specific definitions without changing these meanings.

References

1. IEEE Computer Society, IEEE 830: Recommended Practice for Software Requirements Specifications.
2. OWASP Foundation, OWASP Application Security Verification Standard and OWASP web application security guidance.
3. National Institute of Standards and Technology, Security and Privacy Controls for Information Systems and Organizations (SP 800-53).
4. National Institute of Standards and Technology, Computer Security Incident Handling Guide (SP 800-61).
5. Anti-Phishing Working Group, phishing activity reports and educational resources.
6. Project reference: EXPIROCHAIN report, supplied as a structural reference for academic documentation.
7. Project reference: SRS Report Formatting Guide, supplied for A4 layout, typography, headings, captions, and IEEE-based structure.

End of Document